CforC Website — Security Hardening Report

Date: March 2026 **Scope:** Next.js frontend + API routes, Strapi CMS integration **Status:** All 16 issues addressed (15 fixed, 1 deferred)

Summary

A comprehensive security audit identified 16 vulnerabilities in the CforC website. This document details each issue, its severity, and the fix applied.

#	Issue	Severity	Status
1	Strapi API token exposed in client-side code	Critical	Fixed
2	Hardcoded webhook secret fallback	Critical	Fixed
3	Sensitive data in console.log statements	High	Fixed
4	Magic link tokens not invalidated after use	High	Fixed
5	HTML injection in email templates	High	Fixed
6	Missing rate limiting on 4 endpoints	Medium	Fixed
7	No CSRF protection on POST routes	Medium	Fixed
8	Unencoded user input in Strapi query strings	Medium	Fixed
9	No server-side file upload validation	Medium	Fixed
10	Sensitive fields returned in session endpoint	Medium	Fixed
11	No email uniqueness check on profile update	Medium	Fixed
12	No URL sanitization in rich text rendering	Medium	Fixed
13	In-memory rate limiter (not distributed)	Low	Deferred
14	Missing security headers	Low	Fixed
15	JWT algorithm not explicitly restricted	Low	Fixed
16	Strapi error details leaked to client	Low	Fixed (in #3)

Fix #1 — Remove Strapi API Token from Client-Side Code

Severity: Critical **Risk:** The Strapi API token was exposed via `NEXT_PUBLIC_STRAPI_API_TOKEN`, visible in browser DevTools and JS bundles. Anyone could extract it and read/modify all CMS data.

Changes:

- **New file:** `app/api/strapi/[...path]/route.ts` — Server-side proxy route that forwards client requests to Strapi using the server-only token. Includes an allowlist of permitted collection endpoints (`members`, `activities`, `open-calls`, etc.) and only allows GET requests.

- **Modified:** `lib/strapi.ts` — `fetchStrapi()` now detects server vs client context (`typeof window === 'undefined'`). Server-side calls go directly to Strapi with the token; client-side calls go through the `/api/strapi/...` proxy without any token.
- **Modified:** `app/members/page.tsx`, `app/members/[name]/page.tsx`, `app/members/[name]/[project]/page.tsx` — Replaced direct fetch calls (which used the public token) with `getMembers()` and `getMemberBySlugOrId()` from `lib/strapi.ts`.
- **Modified:** `.env.local` — Removed `NEXT_PUBLIC_STRAPI_API_TOKEN`.

Post-deploy action: The old token was rotated in Strapi Cloud since it had been exposed in client bundles.

Fix #2 — Remove Hardcoded Webhook Secret Fallback

Severity: Critical **Risk:** `app/api/membership/webhook/route.ts` had a hardcoded fallback value (`'your-webhook-secret'`) for the webhook secret. If the environment variable was missing, anyone who guessed this string could forge webhook requests.

Changes:

- **Modified:** `app/api/membership/webhook/route.ts` — Removed the fallback value. If `MEMBERSHIP_WEBHOOK_SECRET` is not set, the endpoint now returns a 500 error instead of silently using an insecure default.

Fix #3 — Remove Sensitive Console.log Statements

Severity: High **Risk:** Several API routes logged sensitive data (JWT tokens, email addresses, member IDs, Strapi error details) to server logs, which could be exposed via log aggregation services.

Changes:

- **Modified:** `app/api/auth/verify-magic-link/route.ts` — Removed 6 console.log statements that logged token values, hashes, email addresses, member IDs, and full auth token data.
- **Modified:** `app/api/members/update/route.ts` — Removed logs exposing the full payload and request body. Removed Strapi error details from client-facing error responses.
- **Modified:** `app/api/subscribe/route.ts` — Removed email addresses from honeypot/suspicious-pattern log messages.

This fix also resolved **#16** (Strapi error details leaked to client).

Fix #4 — Invalidate Magic Link Tokens After Use

Severity: High **Risk:** After a magic link was verified, the token remained valid in the database. An attacker who intercepted a magic link URL could reuse it indefinitely within its expiry window.

Changes:

- **Modified:** `app/api/auth/verify-magic-link/route.ts` — After successful verification, the token record is now deleted from Strapi via a DELETE call to `/api/auth-tokens/{documentId}`.

Fix #5 — HTML Escaping in Email Templates

Severity: High **Risk:** User-supplied data (names, emails, messages) was interpolated directly into HTML email templates without escaping. An attacker could inject malicious HTML/JavaScript into emails sent to administrators.

Changes:

- **Modified:** `app/api/feedback/route.ts` — Added `escapeHtml()` function. All user inputs (name, email, subject, message) are now escaped before insertion into the HTML email body.
- **Modified:** `app/api/subscribe/route.ts` — Email address is escaped in the admin notification email.
- **Modified:** `app/api/working-groups/join/route.ts` — Group name, message body, and user profile URL are escaped in the notification email.

The `escapeHtml()` function replaces `&`, `<`, `>`, `"`, and `'` with their HTML entity equivalents.

Fix #6 — Add Rate Limiting to Unprotected Endpoints

Severity: Medium **Risk:** Four endpoints had no rate limiting, allowing brute-force attacks or abuse (e.g., spamming the feedback form or attempting to guess magic link tokens).

Changes:

- **Modified:** `lib/rateLimiter.ts` — Added 4 new rate limiter instances:
  - `verifyMagicLinkLimiter`: 10 attempts per 15 minutes
  - `setPasswordLimiter`: 5 attempts per 15 minutes
  - `feedbackLimiter`: 5 submissions per hour
  - `workingGroupJoinLimiter`: 5 requests per hour
- **Modified:** `app/api/auth/verify-magic-link/route.ts` — Added `verifyMagicLinkLimiter`
- **Modified:** `app/api/auth/set-password/route.ts` — Added `setPasswordLimiter`
- **Modified:** `app/api/feedback/route.ts` — Added `feedbackLimiter`
- **Modified:** `app/api/working-groups/join/route.ts` — Added `workingGroupJoinLimiter`

Fix #7 — CSRF Protection on All POST Routes

Severity: Medium **Risk:** POST endpoints had no CSRF protection, meaning a malicious website could trick a logged-in user's browser into making state-changing requests (e.g., updating their profile, logging them out).

Changes:

- **New file:** `lib/csrf.ts` — CSRF validation function that checks the `Origin` header (falling back to `Referer`) against an allowlist of permitted origins: `cultureforchange.net`, `www.cultureforchange.net`, and `localhost:3000` (dev only).
- **Modified (9 routes):**
  - `app/api/auth/login/route.ts`
  - `app/api/auth/logout/route.ts`
  - `app/api/auth/request-magic-link/route.ts`
  - `app/api/auth/set-password/route.ts`
  - `app/api/auth/verify-magic-link/route.ts`
  - `app/api/feedback/route.ts`
  - `app/api/members/update/route.ts`
  - `app/api/subscribe/route.ts`

- `app/api/working-groups/join/route.ts`

Each route now calls `checkCsrf(request)` at the start and returns 403 if the origin is not allowed.

Fix #8 — Encode User Input in Strapi Query Strings

Severity: Medium **Risk:** User-supplied values (slugs, IDs, emails) were interpolated directly into Strapi API query strings without encoding. Specially crafted input could manipulate query parameters or cause unexpected API behavior.

Changes:

- **Modified:** `lib/strapi.ts` — Added `encodeURIComponent()` to all 8 slug/ID interpolations in filter query strings across `getMemberBySlugOrId()`, `getActivityBySlug()`, `getOpenCallBySlug()`, and `getProjectEntryBySlug()`.
- **Modified:** `app/api/auth/session/route.ts` — Encoded `decoded.memberId` in the Strapi query.
- **Modified:** `app/api/members/update/route.ts` — Encoded `memberId` and `updateData.Email` in Strapi queries.

Fix #9 — Server-Side File Upload Validation

Severity: Medium **Risk:** The profile update endpoint accepted file uploads without validating file type or size on the server. An attacker could upload arbitrarily large files or non-image files.

Changes:

- **Modified:** `app/api/members/update/route.ts` — Added server-side validation for all uploaded files:
  - **Allowed types:** JPEG, PNG, WebP, GIF, HEIC, HEIF
  - **Maximum size:** 10 MB per file
  - Returns 400 error with descriptive message if validation fails

UX improvement: `app/profile/page.tsx` — Added auto-scroll to error messages when validation fails, using refs on the error display elements.

Fix #10 — Strip Sensitive Fields from Session Endpoint

Severity: Medium **Risk:** The session endpoint returned the full member object including `magicLinkToken` and `magicLinkExpiry` fields. While already stripped of `password`, these additional sensitive fields were exposed.

Changes:

- **Modified:** `app/api/auth/session/route.ts` — Added `magicLinkToken` and `magicLinkExpiry` to the destructured fields that are excluded from the response.

Fix #11 — Email Uniqueness Check on Profile Update

Severity: Medium **Risk:** When a member changed their email address, there was no check for whether another member already used that email. This could lead to duplicate accounts or confusion.

Changes:

- **Modified:** `app/api/members/update/route.ts` — Before updating, the endpoint now queries Strapi to check if any other member (excluding the current one) already has the submitted email. Returns 409 Conflict if a duplicate is found.

Fix #12 — URL Sanitization in Rich Text Rendering

Severity: Medium **Risk:** The rich text renderer (`renderBlocks.tsx`) rendered link URLs directly from Strapi content without sanitization. A content editor (or compromised CMS) could inject `javascript:` URLs that execute code when clicked.

Changes:

- **Modified:** `lib/renderBlocks.tsx` — Added `sanitizeUrl()` function that blocks `javascript:`, `data:`, and `vbscript:` protocol URLs (replacing them with `#`). Applied to all link `href` attributes in rendered content.

Fix #13 — In-Memory Rate Limiter (Deferred)

Severity: Low **Risk:** The rate limiter uses in-memory storage, which resets on server restart and doesn't work across multiple server instances (e.g., Vercel serverless functions).

Status: Deferred. At the current traffic scale, the in-memory approach provides reasonable protection. A distributed solution (Upstash Redis or Vercel KV) would be needed for higher traffic or multi-instance deployments. The risk is low because:

- Vercel function instances have some persistence within a warm period
- The rate limits are still effective against casual abuse
- The auth endpoints also have other protections (token expiry, hashed passwords)

Fix #14 — Security Headers

Severity: Low **Risk:** The site was missing standard security headers that protect against clickjacking, MIME sniffing, and protocol downgrade attacks.

Changes:

- **Modified:** `next.config.js` — Added security headers for all routes:
  - `X-Content-Type-Options: nosniff` — Prevents MIME type sniffing
  - `X-Frame-Options: DENY` — Prevents clickjacking via iframes
  - `Referrer-Policy: strict-origin-when-cross-origin` — Controls referrer information
  - `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` — Forces HTTPS
  - `X-DNS-Prefetch-Control: on` — Enables DNS prefetching for performance
  - `Permissions-Policy: camera=(), microphone=(), geolocation=()` — Disables unused browser APIs

Note: Content-Security-Policy was intentionally omitted as it requires careful tuning for Google Translate widget, Strapi CDN images, and Resend email service.

Fix #15 — JWT Algorithm Restriction

Severity: Low **Risk:** The JWT signing and verification did not explicitly specify the allowed algorithm. While HS256 was used by default, not pinning it leaves a theoretical attack vector where an attacker could manipulate the algorithm header.

Changes:

- **Modified:** `lib/auth.ts` — Explicitly set `algorithm: 'HS256'` in `SignOptions` for both `generateSessionToken()` and `generateMagicLinkToken()`. Added `algorithms: ['HS256']` to the `jwt.verify()` call in `verifyToken()`.

Fix #16 — Strapi Error Details Leaked to Client

Severity: Low **Risk:** Some API routes forwarded raw Strapi error messages to the client, potentially revealing internal infrastructure details.

Status: Already resolved as part of Fix #3. The `members/update` route now returns generic Greek error messages instead of forwarding Strapi error details.

Files Changed (Complete List)

New Files

File	Purpose
`app/api/strapi/[...path]/route.ts`	Strapi API proxy (Fix #1)
`lib/csrf.ts`	CSRF protection utility (Fix #7)

Modified Files

File	Fixes
`lib/strapi.ts`	#1, #8
`lib/auth.ts`	#15
`lib/rateLimiter.ts`	#6
`lib/renderBlocks.tsx`	#12
`next.config.js`	#14
`.env.local`	#1
`app/api/auth/login/route.ts`	#7
`app/api/auth/logout/route.ts`	#7
`app/api/auth/request-magic-link/route.ts`	#7
`app/api/auth/set-password/route.ts`	#6, #7
`app/api/auth/session/route.ts`	#8, #10

File	Fixes
`app/api/auth/verify-magic-link/route.ts`	#3, #4, #6, #7
`app/api/feedback/route.ts`	#5, #6, #7
`app/api/members/update/route.ts`	#3, #7, #8, #9, #11, #16
`app/api/membership/webhook/route.ts`	#2
`app/api/subscribe/route.ts`	#3, #5, #7
`app/api/working-groups/join/route.ts`	#5, #6, #7
`app/members/page.tsx`	#1
`app/members/[name]/page.tsx`	#1
`app/members/[name]/[project]/page.tsx`	#1
`app/profile/page.tsx`	#9 (UX)

Recommendations for Future Work

1. **Content-Security-Policy header** — Should be added once all third-party scripts (Google Translate, analytics) are catalogued
2. **Distributed rate limiting** — Migrate to Upstash Redis or Vercel KV when traffic increases
3. **Strapi API token rotation** — Rotate periodically (quarterly recommended)
4. **Dependency audits** — Run `npm audit` regularly to catch known vulnerabilities in dependencies